

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA5117

COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO3: *Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation.CO (BL4 , BL5)*

Assessment Tool Weightage: 20%

QUESTIONS: 5

TOTAL MARKS: 25

Annexure A CASE STUDY

Code of Conduct:

*I Sivakumar M(192521057) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Sivakumar m
Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

CO 3 AT -2 Case Study

1. A software company is using MD5 to store user passwords in its database. After a security breach, attackers were able to crack multiple passwords. Analyze the weaknesses of MD5 and propose a more secure hashing approach using SHA-based techniques.

Introduction

MD5 (Message Digest Algorithm 5) is a cryptographic hash function that generates a 128-bit hash value. It was widely used for password storage and file integrity verification. However, due to advances in computing power and cryptanalysis, MD5 is no longer considered secure for storing passwords.

Weaknesses of MD5

- Produces only a **128-bit hash**, which is relatively weak by today's standards.
- Vulnerable to **collision attacks**, where two different inputs generate the same hash value.
- Extremely fast, allowing attackers to perform **brute-force** and **dictionary attacks** efficiently.
- If passwords are stored without **salt**, attackers can use **rainbow tables** to recover passwords.
- Does not provide adequate protection against modern password-cracking tools.

Impact of the Security Breach

When the database was compromised, attackers obtained the MD5 password hashes. Because MD5 is fast and vulnerable, attackers used GPU-based password-cracking software and rainbow tables to recover many user passwords. Once passwords were recovered, attackers could access user accounts and potentially perform credential stuffing attacks on other websites.

Secure SHA-Based Solution

A secure password storage method should use:

- **SHA-256 or SHA-512** instead of MD5.
- A unique **random salt** for every password.
- **PBKDF2 with SHA-256**, which performs thousands of hashing iterations.
- Strong password policies and Multi-Factor Authentication (MFA).

Comparison Table

Feature	MD5	SHA-256
Hash Length	128 bits	256 bits
Collision Resistance	Weak	Strong
Brute-force Resistance	Low	High
Recommended Today	No	Yes
Password Storage	Unsafe	Safe (with salt and iterations)

Advantages of SHA-Based Hashing

- Higher security against collision attacks.
- Strong resistance to brute-force attacks.
- Secure when combined with salting and key stretching.
- Recommended by modern cybersecurity standards.

Conclusion

MD5 should never be used for password storage. Replacing MD5 with SHA-256 (or PBKDF2 using SHA-256), along with unique salts and multiple iterations, provides significantly stronger protection against password-cracking attacks.

2. A banking application implements digital signatures for transaction verification, but users report unauthorized transactions being approved. Analyze the possible flaws in the digital signature

implementation and recommend improvements based on secure signature standards.

Introduction

Digital signatures provide authentication, integrity, and non-repudiation in online banking transactions. If unauthorized transactions are approved, it indicates flaws in the digital signature implementation or key management process.

Possible Flaws

- Private keys are stored insecurely.
- Weak hashing algorithms such as SHA-1 are used.
- Signature verification is incomplete or incorrect.
- Expired or revoked certificates are not checked.
- Timestamp validation is missing.
- Users' private keys are compromised through malware.

How Digital Signatures Work

1. User creates a transaction.
2. Transaction data is hashed.
3. The hash is encrypted using the sender's private key.
4. The receiver decrypts the signature using the sender's public key.
5. If both hash values match, the transaction is authentic.

Recommended Improvements

- Use **RSA-3072** or **ECDSA with SHA-256**.
- Store private keys in **Hardware Security Modules (HSMs)**.

- Verify certificate status using **CRL** or **OCSP**.
- Add secure timestamps to transactions.
- Enable Multi-Factor Authentication (MFA).
- Rotate cryptographic keys periodically.

Advantages

- Prevents transaction forgery.
- Ensures authenticity and integrity.
- Protects against key compromise.
- Improves customer trust.

Conclusion

Secure digital signature algorithms, proper certificate validation, secure key storage, and MFA significantly reduce unauthorized transactions in banking applications.

3. An enterprise network uses Kerberos for authentication, but attackers successfully perform a replay attack to gain unauthorized access. Examine how this attack could occur and propose enhancements in the authentication process to prevent such threats.

Introduction

Kerberos is a network authentication protocol developed to provide secure authentication over insecure networks. It uses **secret-key cryptography**, **tickets**, and a trusted third party called the **Key Distribution Center (KDC)**. Kerberos ensures that passwords are never transmitted over the network. However, if authentication tickets are intercepted and reused, attackers may perform a **replay attack**, allowing unauthorized access to enterprise resources.

Kerberos Authentication Process

The Kerberos authentication process consists of the following steps:

1. The user logs into the client computer using valid credentials.
2. The client requests a **Ticket Granting Ticket (TGT)** from the Authentication Server (AS).
3. The Authentication Server verifies the user's identity and issues a TGT.
4. The client presents the TGT to the Ticket Granting Server (TGS) to request access to a service.
5. The TGS issues a **Service Ticket**.
6. The client sends the Service Ticket to the application server.
7. The server validates the ticket and grants access.

This process allows secure authentication without repeatedly sending passwords.

How the Replay Attack Occurs

A replay attack occurs when an attacker captures a valid Kerberos ticket during transmission and retransmits it before it expires.

The attack proceeds as follows:

- The attacker monitors network traffic.

- A valid Kerberos ticket is captured.
- The attacker stores the captured ticket.
- Before the ticket expires, the attacker sends the same ticket to the server.
- If replay protection is weak, the server accepts the ticket and grants access.

Thus, the attacker impersonates a legitimate user without knowing the actual password.

Possible Causes

- Long ticket validity period.
- Lack of replay cache.
- Weak timestamp verification.
- Poor synchronization of system clocks.
- Improper session management.
- Unencrypted communication channels.

Security Enhancements

To prevent replay attacks, organizations should implement:

- Short ticket lifetimes.
- Accurate timestamp verification.
- Network Time Protocol (NTP) synchronization.
- Replay cache mechanisms.
- Mutual authentication between client and server.
- TLS encryption for communication.
- Strong monitoring and logging of authentication events.

Advantages of the Proposed Solution

- Prevents replay attacks.
- Improves authentication security.

- Protects enterprise resources.
- Detects suspicious login attempts.
- Strengthens overall network security.

Conclusion

Although Kerberos is a secure authentication protocol, replay attacks remain possible if ticket management is weak. Using short-lived tickets, replay detection, synchronized clocks, and encrypted communication significantly improves security and prevents unauthorized access.

4. A secure communication system uses a simple hash function to verify message integrity, but attackers are able to modify messages without detection. Analyze why this approach fails and propose a solution using HMAC to improve message authentication.

Introduction

Message integrity ensures that transmitted data has not been altered during communication. A simple cryptographic hash function such as SHA-256 can detect

accidental changes but cannot verify the identity of the sender. Therefore, attackers who intercept a message can modify it and generate a new hash, making the receiver believe the message is genuine.

Why a Simple Hash Function Fails

A hash function converts a message into a fixed-length digest.

Example:

Message

Transfer ₹10,000

Hash

SHA-256(Message)

If an attacker changes the message to

Transfer ₹1,00,000

the attacker can simply compute a new hash and send both the modified message and its new hash.

Since no secret key is involved, the receiver cannot determine whether the message originated from the legitimate sender.

Limitations of Simple Hashing

- No sender authentication.
- Anyone can generate the hash.
- Cannot prevent message forgery.
- Vulnerable to man-in-the-middle attacks.
- Does not guarantee message authenticity.

Using HMAC for Secure Authentication

HMAC (Hash-based Message Authentication Code) combines a secret key with a secure hash function.

Working of HMAC

1. Sender and receiver share a secret key.
2. Sender computes

$\text{HMAC} = \text{HMAC}(\text{Key}, \text{Message})$

using SHA-256.

3. Sender transmits:
 - Original message
 - HMAC value
4. Receiver computes HMAC using the same secret key.
5. If both HMAC values are identical, the message is accepted; otherwise, it is rejected.

Advantages of HMAC

- Provides message integrity.
- Authenticates the sender.
- Prevents unauthorized modification.
- Resistant to replay attacks.
- Resistant to collision-based attacks.
- Widely used in HTTPS, VPNs, APIs, and cloud security.

Comparison

Simple Hash HMAC

No secret key Uses secret key

Integrity only Integrity + Authentication

Easy to forge Very difficult to forge

Less secure Highly secure

Conclusion

Simple hashing alone cannot secure modern communication systems. HMAC with SHA-256 provides strong integrity and authentication, making it the preferred solution for secure communication.

Q5 An organization implements X.509 certificates for secure communication, but users encounter frequent trust validation errors. Analyze the possible issues in certificate management and propose solutions to ensure proper authentication and trust chain validation.

Introduction

X.509 certificates are digital certificates used in SSL/TLS protocols to authenticate users, servers, and organizations. They establish trust through a hierarchical Certificate Authority (CA) system. If certificate validation fails, users receive security warnings and secure communication cannot be established.

Working of X.509 Certificates

The certificate validation process includes:

1. Server sends its X.509 certificate.
2. Client verifies the digital signature.
3. Client checks certificate validity dates.
4. Client verifies the certificate chain.
5. Client checks whether the certificate is revoked.
6. Client verifies that the domain name matches the certificate.
7. If all checks are successful, a secure connection is established.

Possible Causes of Trust Validation Errors

- Expired certificates.
- Self-signed certificates.
- Missing intermediate CA certificates.
- Incorrect hostname.
- Revoked certificates.
- Incorrect system date and time.
- Outdated trusted root certificates.
- Improper certificate installation.

Impact on Organizations

- Users lose trust in the website.
- Secure communication fails.
- Increased risk of phishing attacks.
- Business reputation is affected.
- Possible financial losses.

Recommended Solutions

- Renew certificates before expiration.
- Purchase certificates from trusted Certificate Authorities.
- Install the complete certificate chain.
- Enable CRL and OCSP validation.
- Verify hostname consistency.
- Keep operating systems and browsers updated.
- Automate certificate renewal.
- Continuously monitor certificate health.

Benefits of Proper Certificate Management

- Strong authentication.
- Secure encrypted communication.
- Reduced trust errors.
- Better user confidence.
- Compliance with cybersecurity standards.
- Improved business reputation.

Conclusion

Proper management of X.509 certificates is essential for secure communication. Regular renewal, correct installation, complete trust chain validation, and continuous monitoring eliminate trust validation errors and ensure secure authentication.

